[Your College Logo]

Industrial Internship Report on

”Smart City Traffic Analysis and Crop and Weed detection ”

Prepared by

Palavalasa Kusuma

Executive Summary
This report provides details of the Industrial Internship provided by upskill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT). This internship was focused on a project/problem statement provided by UCT. We had to finish the project including the report in 4 weeks’ time. My project was Smart city traffic analysis and Crop and weed detection This internship gave me a very good opportunity to get exposure to Industrial problems and design/implement solution for that. It was an overall great experience to have this internship.

TABLE OF CONTENTS

1 Preface 3
2 Introduction 4
2.1 About UniConverge Technologies Pvt Ltd 4
2.2 About upskill Campus 8
2.3 Objective 10
2.4 Reference 10
2.5 Glossary 10
3 Problem Statement 12
4 Existing and Proposed solution 13
5 Proposed Design/ Model 15
5.1 High Level Diagram (if applicable) .... **Error! Bookmark not defined.**
5.2 Low Level Diagram (if applicable) .... **Error! Bookmark not defined.**
5.3 Interfaces (if applicable) .... **Error! Bookmark not defined.**
6 Performance Test 16
6.1 Test Plan/ Test Cases 16
6.2 Test Procedure 16
6.3 Performance Outcome 17
7 My learnings 18
8 Future work scope 19

1 Preface

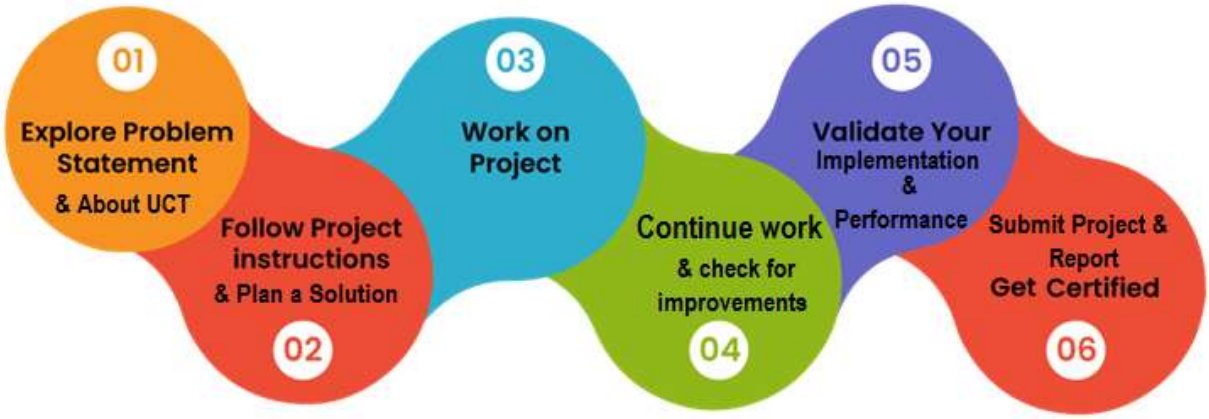
Summary of the whole 6 weeks' work.

About need of relevant Internship in career development.

Brief about Your project/problem statement.

Opportunity given by USC/UCT.

How Program was planned



Your Learnings and overall experience.

Thank to all , who have helped you directly or indirectly.

Your message to your juniors and peers.

2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various **Cutting Edge Technologies e.g. Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front end** etc.

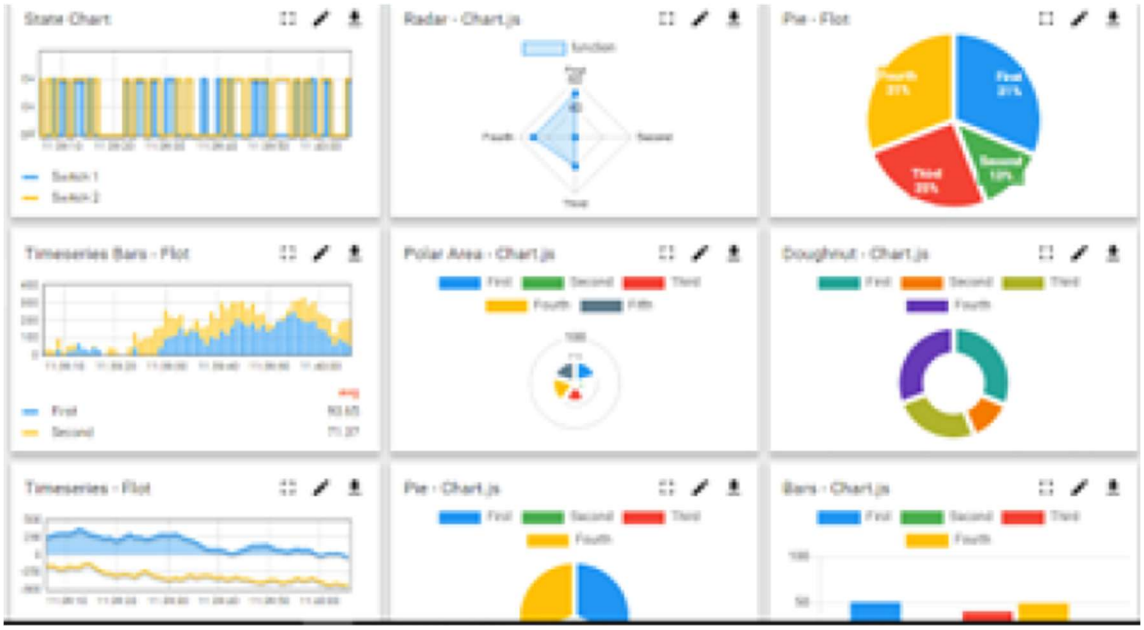
i. UCT IoT Platform (UCT Insight)

UCT Insight is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable "insight" for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA
- It supports both cloud and on-premises deployments.

It has features to

- Build Your own dashboard
- Analytics and Reporting
- Alert and Notification
- Integration with third party application(Power BI, SAP, ERP)
- Rule Engine



ii. Smart Factory Platform (FACTORY WATCH)

Factory watch is a platform for smart factory needs.

It provides Users/ Factory

- with a scalable solution for their Production and asset monitoring
- OEE and predictive maintenance solution scaling up to digital twin for your assets.
- to unleased the true potential of the data that their machines are generating and helps to identify the KPIs and also improve them.
- A modular architecture that allows users to choose the service that they what to start and then can scale to more complex solutions as per their demands.

Its unique SaaS model helps users to save time, cost and money.

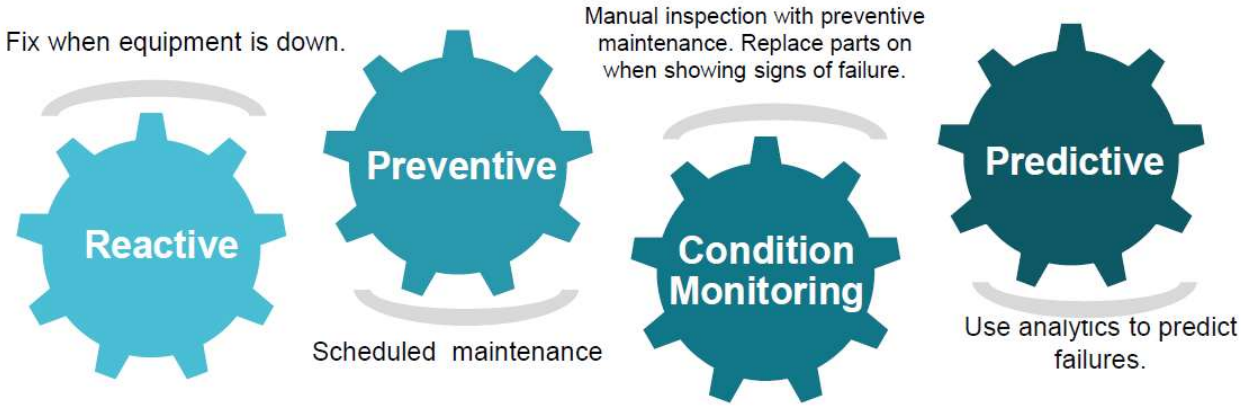
Machine	Operator	Work Order ID	Job ID	Job Performance	Job Progress		Output		Rejection	Time (mins)				Job Status	End Customer
					Start Time	End Time	Planned	Actual		Setup	Pred	Downtime	Idle		
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i

iii. LoRaWAN based Solution

UCT is one of the early adopters of LoRAWAN teschnology and providing solution in Agritech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

iv. Predictive Maintenance

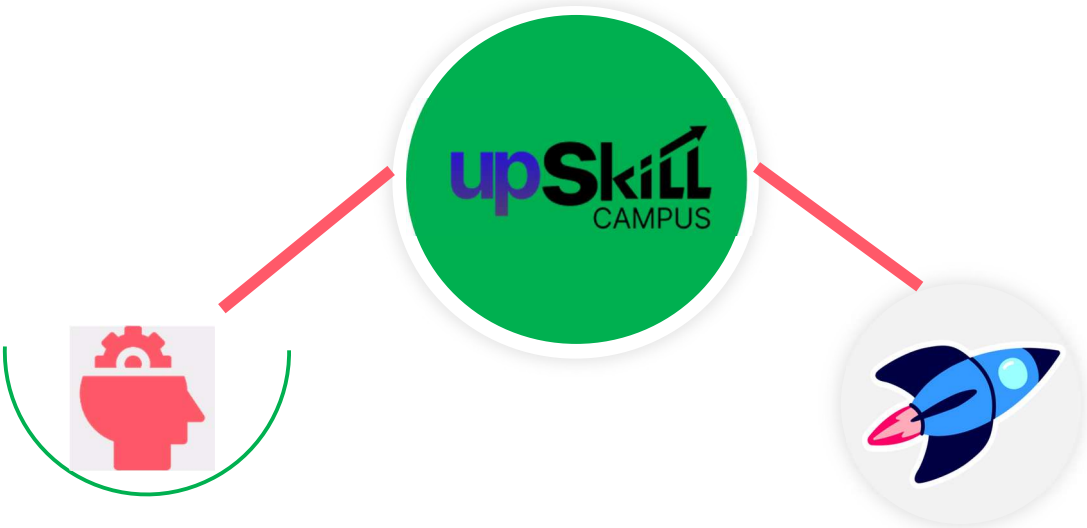
UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.



2.2 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

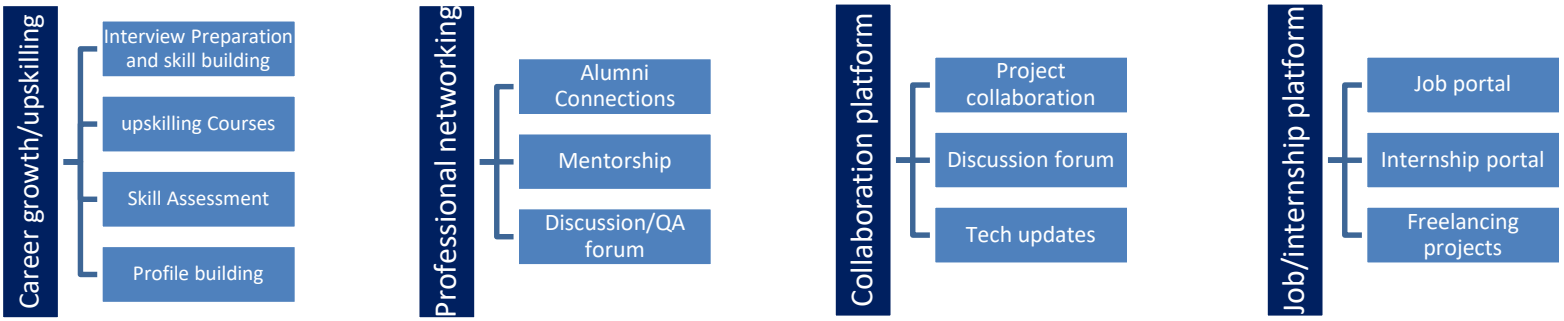
USC is a career development platform that delivers **personalized executive coaching** in a more affordable, scalable and measurable way.



Seeing need of upskilling in self paced manner along-with additional support services e.g. Internship, projects, interaction with Industry experts, Career growth Services

upSkill Campus aiming to upskill 1 million learners in next 5 year

https://www.upskillcampus.com/



2.3 The IoT Academy

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

2.4 Objectives of this Internship program

The objective for this internship program was to

- ☛ get practical experience of working in the industry.
- ☛ to solve real world problems.
- ☛ to have improved job prospects.
- ☛ to have Improved understanding of our field and its applications.
- ☛ to have Personal growth like better communication and problem solving.

2.5 Reference

[1]Kaggle — *Smart City Traffic Patterns*: https://www.kaggle.com/utathya/smart-city-traffic-patterns

[2] Chen, T. & Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System*. KDD '16. — https://arxiv.org/abs/1603.02754

[3]Pedregosa, F. et al. (2011). *Scikit-learn: Machine Learning in Python*. JMLR 12. (used for the Linear Regression baseline and evaluation metrics)

2.6 Glossary

Terms	Acronym
Calendar features	Model inputs derived directly from the timestamp: hour, day-of-week, month, quarter, is-weekend, is-holiday. Capture *recurring* patterns
Cyclical encoding	Representing a periodic value (like hour-of-day) as sin/cos pairs instead of a raw integer, so the model understands hour 23 and hour 0 are adjacent, not far apart.
Feature	The process of turning raw data (a timestamp + a vehicle count) into the richer set of

engineering	inputs a model can actually learn from (lags, rolling stats, calendar flags, trend).
Gradient boosting	A machine learning technique that builds many small decision trees in sequence, where each new tree corrects the errors of the ones before it. XGBoost and LightGBM are both gradient boosting implementations — fast, accurate, and good with tabular/structured data like this.
Holdout / validation set	A slice of data (here, the *last 30 days* of each junction's training history) deliberately withheld from training, used only to check how well the model generalizes before trusting it on real unseen data.

3 Problem Statement

In the assigned problem statement

Smart City — Traffic Pattern Forecasting

You are working with the government to transform your city into a smart city. The vision is to convert it into a digital and intelligent city to improve the efficiency of services for the citizens. One of the problems faced by the government is traffic. You are a data scientist working to manage the traffic of the city better and to provide input on infrastructure planning for the future.

The government wants to implement a robust traffic system for the city by being prepared for traffic peaks. They want to understand the traffic patterns of the four junctions of the city. Traffic patterns on holidays, as well as on various other occasions during the year, differ from normal working days. This is important to take into account for your forecasting.

Crop and Weed detection

Weeds growing alongside sesame crops compete directly with the crop for essential resources such as sunlight, water, and soil nutrients, which leads to reduced yield and lower quality produce. To address this, farmers commonly rely on blanket pesticide spraying, where herbicide is applied uniformly across an entire field regardless of whether weeds are actually present at a given spot. This approach is inefficient and risky in several ways: it results in significant chemical overuse and waste since pesticide is sprayed even on weed-free areas, driving up input costs; it poses a risk to crop health and the environment, as indiscriminate spraying can damage healthy sesame plants and degrade soil quality over time; and it increases health hazards for farm workers due to greater chemical handling and exposure. Currently, there is no low-cost, real-time system capable of visually distinguishing weeds from sesame crop plants in the field and using that distinction to control spraying at a plant-by-plant level. This project therefore aims to design and develop an AI-powered targeted spraying system that uses real-time object detection to differentiate weeds from sesame crops through a live camera feed mounted on a farm vehicle or drone, and selectively triggers pesticide spraying only on detected weed locations. By doing so, the system seeks to reduce chemical usage, protect crop health, minimize human exposure to pesticides, and maintain the detection speed necessary for practical, real-time field operation.

4 Existing and Proposed solution

1. **Fixed-timing traffic signals** — signal cycles set by engineers based on periodic manual surveys, not updated in response to real-time or seasonal changes in traffic volume.
2. **Reactive management** — congestion is addressed only after it becomes visible (complaints, visible gridlock), not anticipated in advance.
3. **Simple statistical forecasting, where forecasting exists at all** — e.g. moving averages or single-model methods like plain ARIMA/SARIMA fit independently per junction, which:
    - Don't account for a **sustained growth trend** (they tend to assume the future looks like a repeat of the recent past, not that traffic keeps growing).
    - Treat **holidays and special events as noise** rather than as a distinct, modelable pattern.
    - Don't validate against a **realistic forecasting scenario** (many naive implementations train and test on randomly shuffled data, which leaks future information and overstates accuracy).
    - Apply the **same method uniformly across all junctions**, even though junctions can behave very differently (e.g. Junction 3's event-driven spikes vs. Junction 1's steady commuter growth).
4. **Siloed data with no forecasting layer** — many cities have sensors/CCTV producing raw vehicle counts, but no pipeline turning that into an actual forward-looking forecast usable for planning.

Limitations of this existing approach: it's descriptive rather than predictive, doesn't scale well across junctions with different data availability (e.g. Junction 4's short 6-month history), and gives planners no quantified, junction-specific answer to "where should we invest first?"

Proposed Solution

This project replaces that with a **data-driven, per-junction forecasting pipeline** with four components:

1. **Pattern discovery (EDA) before modeling** — explicitly quantify growth trend, rush-hour profile, weekday/weekend effect, holiday effect, and event-driven spikes per junction, rather than assuming one pattern fits all four junctions.
2. **Feature-rich, trend-aware modeling** — an ensemble of **XGBoost + LightGBM**, trained per junction, using:
    - Calendar features (hour, day-of-week, holiday flag, cyclical encodings)

- An explicit **trend feature** so the model can extrapolate sustained growth (critical for Junction 1, which tripled in under 2 years)
- Lag & rolling features (1h/24h/1-week history) for short-term momentum

3. **Realistic validation** — a **time-based holdout** (not random shuffling) combined with **recursive forecasting** during validation, so the reported accuracy reflects how the model will actually behave in production, not an inflated number.
4. **Benchmarked against baselines** — every model is compared against a seasonal-naive forecast and a calendar-only linear regression, so the value added by the ensemble is measured and justified, not assumed.
5. **Actionable, dual-audience output** — a technical Jupyter notebook (for data science review/reproducibility) *and* an interactive dashboard (for non-technical government stakeholders), both translating the forecasts into concrete infrastructure recommendations (e.g. prioritize Junction 1 for capacity upgrades, use adaptive signal control rather than scheduling at Junction 3, collect more data before major decisions at Junction 4).

Why this is better: it's proactive (forecasts peaks in advance rather than reacting to them), junction-aware (doesn't force one method onto four different traffic behaviors), honestly validated (no data leakage inflating the numbers), and produces decisions, not just numbers.

4.1 Code submission (Github)

upskillcampus/Crop and Weed Detection at main · Kusuma-97/upskillcampus

upskillcampus/Smart City at main · Kusuma-97/upskillcampus

4.2 Report submission (Github link) :

upskillcampus/SC traffic analysis& crop weed detection_Palavalasa Kusuma_USC_UCT.pdf at main · Kusuma-97/upskillcampus

5 Proposed Design/ Model

Given more details about design flow of your solution. This is applicable for all domains. DS/ML Students can cover it after they have their algorithm implementation. There is always a start, intermediate stages and then final outcome.

6 Performance Test

This is very important part and defines why this work is meant of Real industries, instead of being just academic project.

6.1 Test Plan/ Test Cases

The goal of testing was to check one thing: **can the model actually predict traffic it has never seen, or is it just memorizing the past?**

To test this fairly, I didn't test on random rows — because traffic data is a time series, testing on random rows would let the model accidentally "see the future" while training. Instead:

- For each junction, I held back the **last 30 days** of training data and hid it from the model.
- The model was trained only on data *before* that 30-day window.
- I then asked the model to forecast those 30 days, exactly the way it would forecast the real unseen future — one hour at a time, using its own earlier predictions to help predict later hours (this is called "recursive forecasting").
- I compared the model's forecast against the real, known values for those 30 days.

I tested four different approaches side by side, so I'd know if the fancier model was actually worth using:

1. Seasonal-naive (just guess "same as last week")
2. Linear Regression using only calendar info (day, hour, holiday)
3. XGBoost (full feature set)
4. LightGBM (full feature set)

6.2 Test Procedure

Step by step, for each of the 4 junctions:

1. Split the junction's data into "training" (everything except the last 30 days) and "validation" (the last 30 days).

2. Build all the features (hour, day of week, holiday flag, trend, past values, rolling averages) using only the training portion.
3. Train each of the 4 models on the training data.
4. Forecast the 30 validation days hour-by-hour, feeding each prediction back in as input for the next hour (same method that would be used for the real 4-month forecast).
5. Compare predictions to the real values using two error scores:
    - **MAE** (Mean Absolute Error) — on average, how many vehicles was the forecast off by
    - **RMSE** (Root Mean Squared Error) — similar, but punishes big mistakes more than small ones
6. Repeat this for all 4 junctions and all 4 models, and compare the results in a table.
7. Once I picked the best approach, I retrained it on **all** the training data (no holdout this time) and used it to generate the real forecast for Jul–Oct 2017.

6.3 Performance Outcome

- **XGBoost and LightGBM beat the calendar-only model everywhere** — proving that recent history (not just "what day/hour is it") genuinely helps.
- **At Junctions 1 and 2** (the ones with strong growth), XGBoost/LightGBM clearly beat the naive "same as last week" guess — because naive guessing can't account for traffic still growing.
- **At Junction 3**, naive guessing was surprisingly almost as good as the fancy models — because Junction 3's traffic has sudden random spikes that no model can predict from calendar patterns alone.
- **At Junction 4**, all methods were fairly close — because there's only 6 months of history to learn from, so there isn't much of a trend to learn yet.
- Final choice: a **50/50 blend of XGBoost and LightGBM** — combining two different models usually gives more stable, reliable results than trusting just one.

7 My learnings

- **Traffic data has a strong "memory"** — what happened 1 hour ago and 1 week ago is genuinely useful for predicting what happens next.
- **A growing trend can silently break a forecast** if you don't explicitly tell the model about it — without a trend feature, the model would have kept predicting "old" traffic levels forever.
- **Not every junction behaves the same way.** Testing separately per junction mattered a lot — a method that works great for Junction 1 (steady growth) doesn't work as well for Junction 3 (random spikes).
- **Testing the "right" way matters more than I expected.** If I had tested using randomly shuffled data instead of a proper time-based split, the results would have looked much better than they really are — that would have been misleading.
- **Simple baselines are worth keeping around.** The naive "same as last week" guess wasn't always the worst option — it's a good sanity check that stopped me from overtrusting the fancier models.
- Basic Windows setup issues (broken pip/conda due to a space in the username folder) cost real time — worth checking your environment works *before* starting the real project work.
- **Future Work Scope**
- **Collect more data at Junction 4** — 6 months isn't enough to know its real long-term trend; predictions there are the least trustworthy right now.
- **Add external event data for Junction 3** — if the government has records of festivals, accidents, or road closures, feeding that in directly could explain those sudden spikes instead of treating them as unpredictable noise.
- **Retrain regularly** — traffic patterns keep changing (as we saw with the growth trend), so the model should be retrained every few months with fresh data rather than used as a one-time forecast.
- **Try shorter forecast horizons for real operational use** — this project deliberately forecasted a hard 4-month-ahead scenario; for actual day-to-day traffic management, forecasting just 1 day or 1 week ahead would likely be much more accurate.

8 Future work scope

- **Collect more data at Junction 4** — 6 months isn't enough to know its real long-term trend; predictions there are the least trustworthy right now.
- **Add external event data for Junction 3** — if the government has records of festivals, accidents, or road closures, feeding that in directly could explain those sudden spikes instead of treating them as unpredictable noise.
- **Retrain regularly** — traffic patterns keep changing (as we saw with the growth trend), so the model should be retrained every few months with fresh data rather than used as a one-time forecast.
- **Try shorter forecast horizons for real operational use** — this project deliberately forecasted a hard 4-month-ahead scenario; for actual day-to-day traffic management, forecasting just 1 day or 1 week ahead would likely be much more accurate.
- **Add weather data** — rain, fog, or extreme heat often affects traffic and isn't currently in the model at all.
- **Turn the dashboard into a live, auto-updating tool** — right now it's a snapshot; connecting it to a real-time data feed would make it useful for day-to-day operations, not just planning